

AI Engineering Complete Beginner Roadmap

From zero to production-grade AI systems — step by step

6 Phases

~6 Months

Beginner Friendly

Project-Based

What you will build by the end

- + A RAG-powered Q&A; system with your own documents
- + A memory-enabled AI assistant that remembers users
- + A multi-tool agent using MCP protocol
- + A production API with monitoring, caching, and security

How to use this roadmap

This roadmap is for someone with basic Python knowledge who wants to build production AI systems. Each phase builds on the previous — do not skip phases. The foundations matter more than the fancy tools. Every phase ends with a project. The project is not optional. It is where learning actually happens.

The most important rule

Build something at the end of every phase. Reading tutorials is not learning. A broken project you debugged teaches ten times more than a tutorial you completed perfectly.

Timeline overview

Phase	Topic	Duration	End Goal
1	Python + APIs + Git	2-3 weeks	Call your first LLM API
2	RAG + Embeddings + pgvector	3-4 weeks	Document Q&A system
3	Agents + MCP + LangGraph	3-4 weeks	Multi-tool agent
4	Memory + Evals + Observability	2-3 weeks	Measurable, personalized AI
5	Production + Security + Deployment	3-4 weeks	Live deployed system
6	Advanced RAG + Fine-tuning	3-4 weeks	Optimized, specialized system

Phase 1 — Foundations

Before touching any AI framework you need to be comfortable with Python, APIs, and Git. These are non-negotiable. Every AI tool you will use is a Python library or an HTTP API.

Phase 1

2-3 weeks

Foundations

Python · APIs · Git · First LLM call

Python essentials

Functions, classes, error handling, virtual environments, pip. If you can write a class with methods and handle exceptions you are ready to move on.

FREE COURSE

Python for Everybody — Coursera (Dr. Chuck)

FREE

realpython.com — practical Python tutorials

Working with APIs

HTTP requests, JSON parsing, environment variables for secrets, the requests library. Call a real REST API and save the response before moving on.

FREE

requests library docs — requests.readthedocs.io

PRACTICE

OpenWeatherMap free API — good beginner practice

Git and GitHub

init, add, commit, push, pull, branches. You need this to save your work and show it to employers. Spend two days on this — it pays forever.

FREE

Git and GitHub for Beginners — freeCodeCamp
YouTube

REFERENCE

git-scm.com/doc — official git docs

Your first Claude API call

Get an Anthropic API key. Install the SDK. Send a message. Read the response. Understand what tokens, max_tokens, and the messages format mean.

DOCS

docs.anthropic.com — Messages API quickstart

FREE

Anthropic cookbook —

github.com/anthropics/anthropic-cookbook

Phase 1 project

Build a CLI chatbot. User types a message, your Python script sends it to Claude, prints the reply. Add conversation history so it remembers previous messages in the session. Store your API key in a .env file. Push the project to GitHub.

Phase 2 — RAG and Knowledge

RAG is the most important pattern in AI engineering. It is behind 90% of real AI products. You will learn how to store documents as vectors, search them semantically, and feed the right context to Claude. This is the core skill.

Phase 2

3-4 weeks

RAG and Knowledge

Embeddings · pgvector · Chunking · Hybrid search · Reranking

What are embeddings?

Embeddings convert text into numbers (vectors) that capture meaning. Similar sentences have similar vectors. This is what makes semantic search possible.

WATCH

3Blue1Brown Neural Networks series — YouTube

READ

OpenAI embeddings guide — platform.openai.com/docs

pgvector setup

Install PostgreSQL and the pgvector extension. Store text chunks as vectors. Query with cosine similarity. Understand why this beats a regular database for AI.

DOCS

github.com/pgvector/pgvector — official docs

TUTORIAL

Supabase pgvector guide — supabase.com/docs/guides/ai

Chunking strategies

Split documents into pieces. Fixed-size, sentence-based, paragraph-based. Overlap between chunks. This one decision changes output quality more than anything else.

READ

LlamaIndex chunking guide — docs.llamaindex.ai

PAPER

Lost in the Middle paper — why chunk position matters

Building a full RAG pipeline

Load docs, chunk, embed, store in pgvector. On query: embed query, retrieve top chunks, build prompt with chunks, send to Claude, return answer with sources.

COURSE

LlamaIndex RAG tutorial — docs.llamaindex.ai

FREE

LangChain RAG tutorial — python.langchain.com

Hybrid search

Combine vector search (semantic) with BM25 keyword search. Vector alone misses exact terms. BM25 alone misses meaning. Hybrid beats both.

READ

Weaviate hybrid search explainer — weaviate.io/blog

TOOL

rank_bm25 Python library — `pip install rank-bm25`

Reranking

After retrieval, run a reranker model to reorder chunks by true relevance before sending to Claude. This alone dramatically improves output quality.

DOCS

Cohere Rerank API — cohere.com/rerank

FREE ALT

FlashRank — free local reranker, `pip install flashrank`

Phase 2 project

Build a document Q&A system. Take 20+ PDFs on any topic. Chunk them, embed them, store in pgvector. Build a FastAPI endpoint that accepts a question, retrieves top 5 chunks, and returns Claude's answer with sources cited. Test with questions where the answer is in the docs. Test where it is not — make sure it says it does not know.

Phase 3 — Agents and Tools

Agents are AI systems that decide what to do next rather than following a fixed script. You give them tools and a goal. They figure out the steps. This is where AI gets genuinely powerful — and genuinely tricky to debug.

Phase 3

3-4 weeks

Agents and Tools

Tool use · ReAct loop · MCP · LangGraph · Multi-agent

Tool use and function calling

Claude can call functions you define. You describe the function in JSON schema. Claude decides when to call it and with what arguments. You run it and return the result.

DOCS

docs.anthropic.com — Tool use guide

EXAMPLES

Anthropic cookbook — tool use examples on GitHub

The ReAct loop

Reason, Act, Observe, Repeat. The agent thinks, calls a tool, reads the result, thinks again. Build this manually first before using a framework. Understand the loop deeply.

PAPER

ReAct paper — arxiv.org/abs/2210.03629

TUTORIAL

Simon Willison's blog — build ReAct from scratch

MCP — Model Context Protocol

Anthropic's standard for connecting AI to external tools. Over 1,000 community MCP servers exist. Learn to use existing ones then build your own.

DOCS

modelcontextprotocol.io — official spec and SDKs

SERVERS

github.com/modelcontextprotocol/servers

LangGraph

Build stateful graph-based agent workflows. Define nodes (actions) and edges (decisions). Better than LangChain chains for complex multi-step agents. The 2026 industry standard.

DOCS

langchain-ai.github.io/langgraph

COURSE

LangGraph Academy — free, academy.langchain.com

Agent failure modes

Agents loop, call the wrong tool, hallucinate tool names, or never stop. Add max iterations, tool call validation, fallback handling, and human-in-the-loop steps.

READ

LangSmith agent debugging guide — docs.smith.langchain.com

PRACTICE

Deliberately break your agent and debug it — best teacher

Multi-agent systems

Multiple specialized agents collaborating — one searches, one writes, one reviews. Start simple: two agents with a coordinator. Study CrewAI for the pattern.

DOCS

CrewAI docs — docs.crewai.com

READ

A2A protocol — google.github.io/A2A

Phase 3 project

Build a research agent. Give it three tools: web search, your RAG system from Phase 2, and a calculator. Ask it to research a topic, find relevant documents, synthesize an answer, and cite sources. Add a maximum of 10 tool calls so it cannot loop forever. Log every tool call and result so you can debug what went wrong.

Phase 4 — Memory, Evals, and Observability

Memory makes your system feel personal. Evaluation makes it measurable. Without evals you are guessing whether your system actually works. These are the skills that separate engineers who ship reliable AI from those who only demo well.

Phase 4

2-3 weeks

Memory, Evals, and Observability

Mem0 · RAGAS · LLM-as-judge · Langfuse · Prompt caching

Mem0 — user memory

Mem0 extracts and stores user preferences, facts, and context across sessions. Integrates with your existing stack. Users feel the system knows them.

DOCS

docs.mem0.ai — quickstart guide

GITHUB

github.com/mem0ai/mem0

Specification-driven development

Define system behavior in a spec document before coding. Fixed input/output formats, decision rules, edge case handling. Treat your system prompt like a contract.

READ

Anthropic prompt engineering guide — docs.anthropic.com

CONCEPT

Simon Willison — prompt injection and spec-driven AI

RAGAS — RAG evaluation

The standard framework for measuring RAG quality. Measures faithfulness, answer relevance, context precision, and context recall. Run this on every RAG change you make.

DOCS

docs.ragas.io — RAGAS quickstart

PAPER

RAGAS paper — arxiv.org/abs/2309.15217

LLM-as-judge

Use Claude to evaluate Claude's outputs at scale. Build a golden test set of 50+ questions with expected answers. Run your system, compare with the judge. Automate in CI.

READ

Anthropic evals guide — docs.anthropic.com/evals

TOOL

LangSmith — automatic eval runs on every deploy

Langfuse — observability

Trace every LLM call, tool call, and retrieval step. See exactly what happened when your system gave a bad answer. Start with Langfuse — free, open-source, easy to self-host.

DOCS

langfuse.com/docs — Langfuse quickstart

ALT

smith.langchain.com — LangSmith free tier

Prompt caching

Cache your long system prompt so you do not pay to process it on every request. Anthropic's prompt caching reduces costs by 80-90% on cached tokens. Learn this early.

DOCS

docs.anthropic.com — Prompt caching guide

IMPACT

Saves 80-90% on system prompt token costs per request

Phase 4 project

Add memory and evals to your Phase 2 RAG system. Integrate Mem0 so the assistant remembers user preferences across sessions. Build a test set of 30 questions with known correct answers. Run RAGAS. Change your chunking strategy. Run RAGAS again. Prove with numbers that the change improved or worsened quality.

Phase 5 — Production and Security

Most AI systems never make it to production because the builder did not think about deployment, cost, and security. This phase covers everything needed to ship something real that real users can actually use.

Phase 5

3-4 weeks

Production and Security

FastAPI · Docker · Redis · Rate limiting · Security

FastAPI in depth

Async endpoints, dependency injection, middleware, error handling. Streaming responses with Server-Sent Events so users see output token by token.

DOCS

fastapi.tiangolo.com — full FastAPI tutorial

COURSE

testdriven.io — FastAPI in production

Docker and deployment

Write a Dockerfile, build an image, run locally. Deploy to Railway, Render, or Fly.io — all have free tiers. One command to go from laptop to live URL.

FREE

docker-curriculum.com — Docker for Beginners

DEPLOY

[Railway.app](https://railway.app) — easiest FastAPI deployment

Redis caching

Cache frequent queries so you are not hitting Claude and pgvector for the same question repeatedly. hash(query) as key, response as value. Huge cost and latency savings.

DOCS

redis.io/docs — Redis quickstart

LIBRARY

redis-py — pip install redis

Rate limiting and cost control

Limit requests per user per minute with slowapi. Set token budgets. Add circuit breakers for when Claude's API is slow or rate-limiting you.

LIBRARY

slowapi — rate limiting for FastAPI

READ

[Anthropic rate limit docs](https://docs.anthropic.com) — docs.anthropic.com

Security essentials

Prompt injection defense, secrets management with .env and python-dotenv, PII detection before embedding user data, API authentication with JWT tokens.

READ

OWASP LLM Top 10 —

owasp.org/www-project-top-10-for-llms

LIBRARY

presidio — Microsoft PII detection, pip install

presidio-analyzer

Document pipeline maintenance

When source documents update you need to re-chunk, re-embed, and replace old vectors. Build a pipeline that detects changed files, deletes old vectors, and ingests new ones.

PATTERN

[LlamaIndex IngestionPipeline](https://llama-index) with document tracking

READ

pgvector deletion and upsert patterns

Phase 5 project

Deploy your full system publicly. Containerize with Docker. Deploy to Railway. Add Redis caching, rate limiting (10 requests per minute per user), and JWT authentication. Set up Langfuse for production tracing. Share the URL with someone who has never seen your code and watch them use it. Fix what breaks.

Phase 6 — Advanced RAG and Fine-tuning

Advanced techniques for when the basics are not enough. Do not start here. Earn these tools by proving simpler methods fail first — with RAGAS numbers to back it up.

Phase 6

3-4 weeks

Advanced RAG and Fine-tuning

GraphRAG · HyDE · Context engineering · LoRA · Structured outputs

GraphRAG

Build a knowledge graph from your documents instead of chunks. Traverse relationships between entities. Better for complex multi-hop questions.

LIBRARY

github.com/microsoft/graphrag

ALT

LlamaIndex KnowledgeGraphIndex

HyDE — Hypothetical Document Embeddings

Ask Claude to write a hypothetical perfect answer, embed that, then search. The hypothetical answer is closer to document space than the raw query. Often improves recall.

PAPER

HyDE paper — arxiv.org/abs/2212.10496

CODE

5-line implementation available in LangChain

Context engineering

Design the entire information pipeline. What gets retrieved, how it is formatted, what conversation history is kept vs summarized, token budget allocation across components.

READ

Anthropic context window management guide

TOOL

tiktoken — count tokens before sending, pip install tiktoken

LoRA fine-tuning

When RAG and prompting cannot fix a consistent failure pattern, fine-tune. Use Llama 3 8B as the base. Curate 500-1000 instruction-response pairs. Train with trl.

LIBRARY

github.com/huggingface/trl — fine-tuning library

COURSE

Hugging Face fine-tuning course — huggingface.co/learn

Structured outputs

Force Claude to return validated JSON schemas using the Instructor library. Reliable structured data instead of free text. Essential for downstream processing.

LIBRARY

github.com/jxn1/instructor — structured LLM outputs

DOCS

Anthropic tool use for structured output

Multi-tenancy and metadata filtering

Isolate each customer's data in pgvector using metadata filters. Add user permission levels to chunk metadata and filter at retrieval time.

DOCS

pgvector — WHERE clause on metadata JSON columns

PATTERN

LlamaIndex metadata filtering guide

Phase 6 project

Pick the weakest part of your system as measured by RAGAS. Apply one advanced technique to fix it. Measure before and after. Write a short document explaining what you tried, what the numbers showed, and what you would try next. This is exactly the kind of thinking that gets you hired.

Master resource list

Courses and structured learning

FREE	fast.ai — Practical Deep Learning for Coders (builds deep intuition)
FREE	DeepLearning.AI short courses — learn.deeplearning.ai (2-4 hrs each)
FREE	LangChain Academy — academy.langchain.com
FREE	Hugging Face NLP course — huggingface.co/learn/nlp-course
PAID	Udemy: AI Engineer Bootcamp 2026 — comprehensive end-to-end course

Documentation to read deeply

ESSENTIAL	docs.anthropic.com — Claude API, tools, caching, evals
ESSENTIAL	docs.ragas.io — RAG evaluation framework
ESSENTIAL	langchain-ai.github.io/langgraph — stateful agents
ESSENTIAL	modelcontextprotocol.io — MCP protocol and building servers
ESSENTIAL	docs.mem0.ai — user memory layer

Books

BOOK	Designing Machine Learning Systems — Chip Huyen (O'Reilly)
BOOK	Building LLM Apps — Valentina Alto (Packt)
FREE	Patterns for Building LLM-based Systems — Eugene Yan (blog series)

YouTube channels

CHANNEL	Andrej Karpathy — deep intuition for how LLMs actually work
CHANNEL	AI Jason — practical agent building tutorials
CHANNEL	Sam Witteveen — Claude and LangChain tutorials
CHANNEL	Two Minute Papers — stay updated on AI research

Newsletters and communities

NEWSLETTER	The Pragmatic Engineer — pragmaticengineer.com (industry trends)
NEWSLETTER	TLDR AI — daily AI news digest, free
COMMUNITY	LangChain Discord — help with agents and frameworks
COMMUNITY	Hugging Face Discord — open-source models and fine-tuning
FOLLOW	Twitter/X: @karpathy, @simonw, @swyx, @jerryliu0

The honest truth about learning AI engineering

You will be confused. That is normal.

Every engineer building AI systems is figuring things out as they go. The field is 2-3 years old in its current form. There are no 10-year veterans of LangGraph. Confusion is not a sign you are doing it wrong. It is a sign you are doing it.

Tutorials are a trap.

A tutorial that works perfectly teaches you the happy path. Production is nothing but edge cases. The moment you deviate from the tutorial — different documents, different questions, different scale — things break. That breaking is your actual education. Do not spend more than one week on any tutorial before building something of your own.

The stack will change. The principles will not.

LangChain, LlamaIndex, Mem0 — specific tools come and go. But chunking well, evaluating rigorously, building agents that fail gracefully, shipping to production — these are stable. Learn the why behind each tool, not just the how.

One good project beats ten tutorials on a resume.

Build the Phase 2 document Q&A; system with real documents, real evals, and a real deployment. Write a one-page document explaining your chunking decisions and what the RAGAS numbers showed. That one project puts you ahead of 80% of applicants who list RAG as a skill without being able to explain what a reranker is.

Start today. Not when you feel ready.

You will never feel ready. The correct time to start Phase 1 is immediately after reading this. Open a terminal. Install Python. Get an Anthropic API key. Write your first API call. The roadmap is in your hands. The rest is execution.

This roadmap covers everything discussed — the architecture, the new tools, the production gaps, and the advanced techniques. Phase by phase, project by project, you will build the system you described.

Good luck.